



1 Обязательное задание

2 Варианты

3 Нормы

4 Вопросы для самопроверки

5 Факультативное задание

Лабораторная работа 3. Разработка класса

1 Обязательное задание

Требования ко всем с лабораторным работам

1. Написать класс согласно варианту.
 - Помимо конкретных требований варианта, предусмотреть в классе конструктор с параметрами и get-методы для доступа к полям (состоянию) объекта.
 - Объявление/определение класса должно быть вынесено в отдельный модуль.
2. Написать функции (функцию) над контейнером (вектором объектов) согласно варианту.
 - Функция должна принимать вектор объектов и дополнительные параметры при необходимости.
 - Если функция ищет объект, то она должна вернуть индекс найденного объекта или -1 в случае неудачи.
3. Если в варианте явно не указано другое, то в `main` реализовать консольный интерфейс для работы с набором объектов написанного класса в виде меню со следующими пунктами:
 - вставка нового элемента в контейнер (запрашивается индекс вставки и необходимые данные для создания объекта);
 - удаление элемента из контейнера (запрашивается индекс удаляемого элемента);
 - вывод содержимого контейнера в стандартный поток вывода (если вывод содержимого контейнера происходит каждый раз при выводе меню, то оформлять отдельный пункт меню для вывода не требуется);
 - вызов функции над контейнером согласно варианту и вывод найденного объекта (или вывод сообщения об его отсутствии)
 - если таких функций несколько, каждая функция оформляется в виде отдельного пункта меню.

Возможный пример рабочей сессии для 7-го варианта:

ЛР №: 3
Группа: 6111
Автор: Иванов Иван
Вариант: 7

Степенные функции $a \cdot x^b$: []
[1] Вставить новую степенную функцию
[2] Удалить степенную функцию
[3] Найти степенную функцию с минимальным значением производной
[4] Выход

Выберите пункт меню: 1
Введите индекс вставки: 0
Введите параметры a и b: 1 2

Степенные функции: [$1 \cdot x^2$]
[1] Вставить новую степенную функцию
[2] Удалить степенную функцию
[3] Найти степенную функцию с минимальным значением производной
[4] Выход

Выберите пункт меню: 1
Введите индекс вставки: 1
Введите параметры a и b: 3 4

Степенные функции: [$1 \cdot x^2$; $3 \cdot x^4$]
[1] Вставить новую степенную функцию
[2] Удалить степенную функцию
[3] Найти степенную функцию с минимальным значением производной
[4] Выход

Выберите пункт меню: 3
Введите значение аргумента x: 1
Функция с минимальным значением производной:
- Индекс функции в контейнере: 0
- Вид функции: $1 \cdot x^2$

- Значение функции в точке $x=1$: 1
- Значение производной функции в точке $x=1$: 2

Степенные функции: $[1 \cdot x^2; 3 \cdot x^4]$

- [1] Вставить новую степенную функцию
- [2] Удалить степенную функцию
- [3] Найти степенную функцию с минимальным значением производной
- [4] Выход

Выберите пункт меню: 4

До свидания!

2 Варианты

1. Вкладовый счёт

- Состояние:
 - ФИО владельца счета;
 - текущий баланс;
 - % годовой ставки.
- Поведение:
 - начисление процентов за расчётный месяц в виде увеличения текущего баланса на величину $\text{текущий баланс} * \%_{\text{годовой_ставки}} / 12$.
- Функции над контейнером:
 - начислить проценты по всем счетам;
 - найти счёт с максимальным текущим балансом.

2. Пенсионёр

- Состояние
 - СНИЛС;
 - ФИО;
 - стаж.
- Поведение:
 - расчёт социальной выплаты по заданному значению МРОТ:
 - 0, если стаж меньше 5 лет;
 - от $1 * \text{МРОТ}$ до $4 * \text{МРОТ}$ при стаже от 5 до 30 лет (зависимость линейная);
 - $5 * \text{МРОТ}$ при стаже более 30 лет.
- Функция над контейнером:

- найти пенсионера с наибольшей социальной выплатой при заданном значении МРОТ.

3. Штатный сотрудник

- Состояние:
 - ФИО;
 - дата наёма на работу;
 - базовая ставка за 1 месяц работы.
- Поведение:
 - расчёт заработной платы на указанную дату, размер которой определяется базовой ставкой с надбавкой 0.5 % за каждый год работы в организации.
- Функция над контейнером:
 - найти сотрудника с максимальной заработной платой.

4. Круг

- Состояние:
 - центр (предусмотреть вспомогательный класс `Point`);
 - радиус.
- Поведение:
 - вычисление периметра;
 - вычисление площади.
- Функция над контейнером:
 - найти круг с максимальной площадью.

5. Трапеция (с нижней и верхней стороной, параллельными осям координат)

- Состояние:
 - *определить самостоятельно* (предусмотреть вспомогательный класс `Point`).
- Поведение:
 - вычисление периметра;
 - вычисление площади.
- Функция над контейнером:
 - найти трапецию с минимальной площадью.

6. Цилиндр в произвольной ориентации

- Состояние:
 - центры оснований (предусмотреть вспомогательный класс `Point`);
 - радиус.
- Поведение:

- вычисление площади поверхности;
- вычисление объёма.
- Функция над контейнером:
 - найти цилиндр с максимальным объёмом.

7. Степенная функция $f(x)=a \cdot x^b$

- Состояние:
 - параметры функции.
- Поведение:
 - вычисление значения функции от заданного аргумента `x` (допускается реализовать в виде оператора `()`);
 - получение производной функции (напоминание: производная функции — это тоже функция!).
- Функция над контейнером:
 - найти функцию с минимальным значением производной при заданном аргументе `x`.

8. Гармоническая функция $f(x)=a \cdot \cos(\omega \cdot x + \phi)$

- Состояние:
 - параметры функции.
- Поведение:
 - вычисление значения функции от заданного аргумента `x` (допускается реализовать в виде оператора `()`);
 - получение производной функции (напоминание: производная функции — это тоже функция!).
- Функция над контейнером:
 - найти функцию с максимальным значением при заданном аргументе `x`.

9. Геометрическая прогрессия

- Состояние:
 - первый член прогрессии;
 - знаменатель прогрессии.
- Поведение:
 - вычисление `n`-го элемента последовательности по заданному `n`;
 - вычисление суммы первых `n` элементов последовательности при заданном `n`.
- Функция над контейнером:
 - найти прогрессию с максимальной суммой первых `n` элементов при заданном `n`.

10. Последовательность квадратов натуральных чисел с произвольным целым

сдвигом c : $x(n)=n^2+c$

- Состояние:
 - величина сдвига c .
- Поведение:
 - вычисление n -го элемента последовательности при заданном n (допускается реализовать в виде оператора $()$).
- Функция над контейнером:
 - найти последовательность с минимальным n -ым членом при заданном n .

11. Последовательность чисел, кратных c : $x(n)=\{c, 2c, 3c, 4c, \dots\}$, где $c \in \mathbb{N}$

- Состояние:
 - параметр c .
- Поведение:
 - вычисление n -го элемента последовательности при заданном n (допускается реализовать в виде оператора $()$).
- Функция над контейнером:
 - найти последовательность с максимальным n -ым членом при заданном n .

12. Грузовой автомобиль

- Состояние:
 - название модели;
 - объем двигателя v ;
 - тоннаж t .
- Поведение:
 - расчёт транспортного налога по формуле $s \cdot v \cdot (t/2 + 1)$ при заданной базовой ставке s ;
- Функция над контейнером:
 - найти грузовой автомобиль с наибольшим значением транспортного налога.

13. Воздушный транспорт

- Состояние:
 - название модели;
 - тип двигателя (турбовинтовой или реактивный).
- Поведение:
 - расчёт цены транспортировки:

- входные аргументы:
 - базовый тариф перевозки k [руб/(кг·км)];
 - масса груза m [кг];
 - расстояние d [км].
- алгоритм расчёта:
 - $k \cdot m \cdot d \cdot c$, где:
 - для турбовинтового транспорта:
 - $c=0.75$, если $d < 1000$;
 - $c=1.25$, иначе.
 - для реактивного транспорта:
 - $c=1.25$, если $d < 1000$;
 - $c=0.75$, иначе.
- Функция над контейнером:
 - найти транспорт с минимальной ценой транспортировки при заданных k , m и d .

14. (*) Пешка (шахматная фигура)

- Состояние:
 - цвет фигуры;
 - позиция;
 - ориентация (может ходить вверх или вниз?);
 - первый ли это ход.
- Поведение:
 - без учёта других фигур на доске (метод без параметров):
 - проверка возможности пешки переместиться на указанные координаты;
 - проверка возможности пешки атаковать на указанные координаты.
 - с учётом других фигур на доске (метод с параметром типа вектор пешек):
 - проверка возможности фигуры переместиться на указанные координаты;
 - проверка возможности фигуры атаковать на указанные координаты.
- Функция над контейнером:
 - найти фигуру, которая может переместиться на указанные координаты.

15. (*) Ферзь (шахматная фигура)

- Состояние:
 - цвет фигуры;
 - позиция.
- Поведение:
 - без учёта других фигур на доске (метод без параметров):

- проверка возможности пешки переместиться на указанные координаты;
- проверка возможности пешки атаковать на указанные координаты.
- с учётом других фигур на доске (метод с параметром типа вектор ферзей):
 - проверка возможности фигуры переместиться на указанные координаты;
 - проверка возможности фигуры атаковать на указанные координаты.
- Функция над контейнером:
 - найти фигуру, которая может атаковать на указанные координаты.

16. (*) Директория файловой системы (упрощённая симуляция [inode](#))

- Состояние:
 - уникальный идентификатор (число — аналог номера индексного дескриптора; идентификатор не изменяется на протяжении всей жизни объекта);
 - имя директории;
 - опциональный идентификатор родительской директории.
- Поведение:
 - построение абсолютного пути при заданном наборе всех директорий.
 - Используемый контейнер для представления набора всех директорий выбрать самостоятельно и обосновать свой выбор.
 - Пусть есть следующий набор директорий:

ID	Name	ParentID
1	""	—
2	"home"	1
3	"sergey"	2

- Тогда при построении абсолютного пути должны получиться следующие значения:
 - для директории с id=1: `/`;
 - для директории с id=2: `/home`;
 - для директории с id=3: `/home/sergey`;
- Функция над контейнером:
 - поиск идентификатора директории по заданному абсолютному пути.

17. (*) Генератор псевдослучайных чисел `LCG`

См. [линейный конгруэнтный метод \(LCG\)](#).

- Состояние:

- *определить самостоятельно.*
- Поведение:
 - вычисление следующего случайного числа (допускается реализовать в виде оператора `()`).
- Функция над контейнером:
 - поиск ГПСЧ с наибольшим следующим генерируемым числом.
 - Опционально: модифицируйте класс таким образом, чтобы можно было реализовать этот метод без изменения состояния ГПСЧ в контейнере.

18. (*) Персонаж компьютерной игры

- Состояние:
 - количество очков жизни;
 - количество очков брони;
 - количество наносимого урона;
 - вероятность снизить получаемый урон вдвое;
 - ГПСЧ и `uniform_real_distribution` для расчёта шанса снизить получаемый урон вдвое.
- Поведение:
 - принять входящий урон (параметр метода — количество входящего урона `input_damage`);
 - входящий урон `input_damage` уменьшается в зависимости от очков брони (конкретную формулу расчёта придумать самостоятельно);
 - кроме того, в зависимости от вероятности снизить получаемый урон вдвое, входящий урон может быть дополнительно уменьшен в два раза.
 - применить “особое умение”, которое увеличивает собственную броню, но ослабляет наносимый урон (конкретные формулы расчёта придумать самостоятельно).
- В `main` создать двух рыцарей. Далее либо смоделировать бой полностью, либо моделировать только одного персонажа (вторым при этом управляет пользователь).
 - При моделировании за один раунд случайно выбирается либо атака, либо использование особого умения, если оно не находится на восстановлении.
- Все конкретные числовые значения величин (игровой баланс), включая время восстановления особого умения, подобрать самостоятельно.

3 Нормы

См. раздел [Classes and class hierarchies](#) в части, НЕ касающейся иерархии классов.

4 Вопросы для самопроверки

- Что такое исключения?
- С помощью какой синтаксической конструкции исключение генерируется?
- С помощью какой синтаксической конструкции исключение перехватывается?
- Что такое конструктор?
- Что такое конструктор по умолчанию?
- Что такое конструктор копирования?
- Каким должно быть предпочтительное поведение конструктора по умолчанию?
- Что такое список инициализации в конструкторе?
- Зачем нужно ключевое слово `explicit` при объявлении конструктора?
- Для чего нужны синтаксические конструкции `= default` и `= delete` при объявлении конструктора?

5 Факультативное задание

Факультативная часть засчитывается только при сдаче лабораторной работы на операционной системе семейства Linux.

1. Вынести класс и функцию над контейнером в отдельную статическую библиотеку.
 - Выполнение данного пункта никак не влияет на исходный код программы. все изменения делаются исключительно в файле `CMakeLists.txt`.
 - Данный пункт не сопровождается какой-либо теорией или примерами и подразумевает самостоятельную работу с документацией CMake. Некоторые CMake-функции, которые понадобятся для выполнения задания:
 - `add_library`;
 - `target_link_libraries`.
 - Не стоит перемешивать файлы двух разных целей сборки (библиотеки и основного приложения) в одной директории. Разместите их в отдельных поддиректориях.
2. Добавьте третью цель сборки — приложение с модульными тестами для вашего класса с использованием [GoogleTest](#).
 - Тестами должны быть покрыты только содержательные методы класса с нетривиальной логикой. Конструкторы и get/set-методы тестами не покрываются.